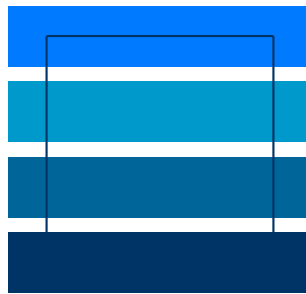


SQL: Guía Completa

Structured Query Language

Edición 2026



Informe Profesional Técnico

Departamento de Bases de Datos

Marzo 2026

Contents

1	Introducción a SQL	3
1.1	Importancia de SQL en 2026	3
1.2	Sistemas de Gestión que Usan SQL	3
2	Conceptos Fundamentales	3
2.1	Bases de Datos Relacionales	3
2.2	Elementos Clave	4
2.3	Tipos de Datos Comunes	4
3	Tipos de Comandos SQL	4
3.1	DDL - Data Definition Language	5
3.2	DML - Data Manipulation Language	5
3.3	DCL - Data Control Language	6
3.4	TCL - Transaction Control Language	6
4	Consultas y Operadores	6
4.1	Cláusulas Fundamentales	7
4.2	Operadores de Comparación y Lógicos	7
4.3	Funciones de Agregación	7
5	JOINS y Uniones de Tablas	8
5.1	Tipos de JOIN	8
5.2	Ejemplos Prácticos de JOIN	9
5.3	Optimización de JOINS	9
6	Subconsultas	9
6.1	Tipos de Subconsultas	10
6.2	Subconsultas vs JOINS	10
7	Funciones SQL	10
7.1	Funciones de Cadena	11
7.2	Funciones Numéricas	11
7.3	Funciones de Fecha y Hora	11
7.4	Funciones de Ventana (Window Functions)	12
8	Vistas e Índices	12
8.1	Vistas (Views)	12
8.2	Índices (Indexes)	13
9	Stored Procedures y Triggers	14
9.1	Stored Procedures	14
9.2	Triggers	15
10	Optimización de Consultas	16
10.1	Principios Fundamentales	16
10.2	Técnicas de Optimización Prácticas	17
10.3	EXPLAIN y Planes de Ejecución	17

11 Transacciones y Propiedades ACID	18
11.1 Propiedades ACID	18
11.2 Ejemplo de Transacción Bancaria	18
11.3 Niveles de Aislamiento	19
12 SQL Avanzado	19
12.1 Common Table Expressions (CTE)	19
12.2 PIVOT - Transformar Filas a Columnas	19
13 Mejores Prácticas y Seguridad	20
13.1 Diseño de Base de Datos	20
13.2 Seguridad SQL	20
14 Conclusiones	21
A Recursos de Aprendizaje	21
A.1 Plataformas Recomendadas	21
A.2 Ruta de Aprendizaje Sugerida	21
B Referencias Bibliográficas	22

1 Introducción a SQL

Definición

SQL (Structured Query Language) es el lenguaje estándar ANSI/ISO para gestionar bases de datos relacionales. Permite crear, leer, actualizar y eliminar datos de manera estructurada y eficiente.

1.1 Importancia de SQL en 2026

A pesar de la emergencia de tecnologías NoSQL y big data, SQL mantiene su relevancia:

Información Clave

Estadísticas Clave:

- **73%** de trabajos en datos requieren SQL como habilidad principal
- Profesionales SQL ganan **15-30%** más que sus contrapartes
- Más de **50 años** de existencia y continua evolución
- Compatible con **todos** los principales sistemas de bases de datos

1.2 Sistemas de Gestión que Usan SQL

Table 1: Principales Sistemas de Bases de Datos Relacionales

Sistema	Desarrollador	Licencia
MySQL	Oracle Corporation	Open Source / Comercial
PostgreSQL	PostgreSQL Global Development Group	Open Source
SQL Server	Microsoft	Comercial
Oracle Database	Oracle Corporation	Comercial
SQLite	D. Richard Hipp	Open Source
MariaDB	MariaDB Foundation	Open Source

2 Conceptos Fundamentales

2.1 Bases de Datos Relacionales

Una **base de datos relacional** organiza datos en tablas interconectadas mediante relaciones definidas por claves.



Figure 1: Relación entre tablas mediante claves foráneas

2.2 Elementos Clave

Table 2: Conceptos Fundamentales de SQL

Término	Definición
Tabla	Estructura que almacena datos en filas y columnas
Columna/Campo	Atributo específico de una tabla (ej: nombre, salario)
Fila/Registro	Conjunto de datos relacionados que representan una entidad
Primary Key (PK)	Columna(s) que identifica únicamente cada registro
Foreign Key (FK)	Columna que referencia a la PK de otra tabla
Índice	Estructura que acelera la recuperación de datos
Vista	Tabla virtual basada en una consulta almacenada

2.3 Tipos de Datos Comunes

Table 3: Tipos de Datos en SQL

Categoría	Tipo	Descripción
3*Numéricos	INT	Entero
	DECIMAL(10,2)	Decimal con precisión
	FLOAT	Punto flotante
3*Texto	VARCHAR(50)	Texto longitud variable
	TEXT	Texto largo
	CHAR(10)	Texto longitud fija
3*Fecha/Hora	DATE	Fecha (YYYY-MM-DD)
	TIME	Hora (HH:MM:SS)
	DATETIME	Fecha y hora combinadas
Booleano	BOOLEAN	TRUE/FALSE

3 Tipos de Comandos SQL

SQL se organiza en cuatro categorías principales de comandos:

Table 4: Categorías de Comandos SQL

Categoría	Comandos	Propósito
DDL	CREATE, ALTER, DROP, TRUNCATE	Definir estructura
DML	INSERT, SELECT, UPDATE, DELETE	Manipular datos
DCL	GRANT, REVOKE	Controlar acceso
TCL	COMMIT, ROLLBACK, SAVEPOINT	Gestionar transacciones

3.1 DDL - Data Definition Language

Los comandos DDL definen la estructura de la base de datos.

Ejemplo Práctico

Creación de Tabla con Constraints:

```

1 CREATE TABLE empleados (
2     id INT PRIMARY KEY AUTO_INCREMENT,
3     nombre VARCHAR(100) NOT NULL,
4     email VARCHAR(100) UNIQUE,
5     salario DECIMAL(10,2) CHECK (salario > 0),
6     fecha_contratacion DATE DEFAULT CURRENT_DATE,
7     depto_id INT,
8     FOREIGN KEY (depto_id) REFERENCES departamentos(id)
9 );

```

Comandos ALTER comunes:

- ALTER TABLE empleados ADD COLUMN telefono VARCHAR(20);
- ALTER TABLE empleados MODIFY COLUMN email VARCHAR(150);
- ALTER TABLE empleados DROP COLUMN telefono;

3.2 DML - Data Manipulation Language

Los comandos DML manipulan los datos dentro de las tablas.

Ejemplo Práctico

Operaciones CRUD Completas:

```

1  -- CREATE (Insertar)
2  INSERT INTO empleados (nombre, email, salario, depto_id)
3  VALUES ('Ana_Garc a', 'ana@empresa.com', 55000.00, 1);
4
5  -- READ (Consultar)
6  SELECT nombre, salario FROM empleados WHERE salario > 50000;
7
8  -- UPDATE (Actualizar)
9  UPDATE empleados
10 SET salario = salario * 1.1
11 WHERE depto_id = 2;
12
13 -- DELETE (Eliminar)
14 DELETE FROM empleados WHERE salario < 30000;

```

3.3 DCL - Data Control Language

Controlan el acceso a los datos.

```

1  -- Conceder permisos
2  GRANT SELECT, INSERT ON empleados TO usuario1;
3  GRANT ALL PRIVILEGES ON empresa.* TO administrador;
4
5  -- Revocar permisos
6  REVOKE INSERT ON empleados FROM usuario1;
7  REVOKE ALL PRIVILEGES ON empresa.* FROM usuario2;

```

3.4 TCL - Transaction Control Language

Gestionan transacciones para garantizar integridad.

```

1  START TRANSACTION;
2
3  UPDATE cuentas SET saldo = saldo - 100 WHERE id = 1;
4  UPDATE cuentas SET saldo = saldo + 100 WHERE id = 2;
5
6  -- Confirmar cambios
7  COMMIT;
8
9  -- O deshacer cambios
10 -- ROLLBACK;

```

4 Consultas y Operadores

4.1 Cláusulas Fundamentales

```

1 SELECT [columnas]           -- Qu  columnas seleccionar
2 FROM [tabla]               -- De qu  tabla
3 WHERE [condici n]          -- Filtrar filas
4 GROUP BY [columnas]        -- Agrupar resultados
5 HAVING [condici n_grupo]    -- Filtrar grupos
6 ORDER BY [columnas]        -- Ordenar resultados
7 LIMIT [n mero];           -- Limitar resultados

```

4.2 Operadores de Comparación y Lógicos

Table 5: Operadores SQL

Tipo	Operadores	Ejemplo
Comparación	=, !=, >, <, ≥, ≤	salario > 50000
Lógicos	AND, OR, NOT	depto_id = 1 AND salario > 40000
Rango	BETWEEN	salario BETWEEN 40000 AND 60000
Lista	IN	depto_id IN (1, 2, 3)
Patrón	LIKE	nombre LIKE 'A%'
Nulo	IS NULL	telefono IS NULL

4.3 Funciones de Agregación

Table 6: Funciones de Agregación

Función	Descripción	Ejemplo
COUNT()	Cuenta registros	SELECT COUNT(*) FROM empleados;
SUM()	Suma valores	SELECT SUM(salario) FROM empleados;
AVG()	Calcula promedio	SELECT AVG(salario) FROM empleados;
MIN()	Valor mínimo	SELECT MIN(salario) FROM empleados;
MAX()	Valor máximo	SELECT MAX(salario) FROM empleados;

Ejemplo Práctico

Consulta con Agregación y Agrupamiento:

```

1 SELECT
2     depto_id,
3     COUNT(*) as total_empleados,
4     AVG(salario) as salario_promedio,
5     MAX(salario) as salario_maximo
6 FROM empleados
7 GROUP BY depto_id
8 HAVING AVG(salario) > 45000
9 ORDER BY salario_promedio DESC;
```

5 JOINS y Uniones de Tablas

Definición

Un **JOIN** combina filas de dos o más tablas basándose en una columna relacionada entre ellas.

5.1 Tipos de JOIN

Table 7: Tipos de JOIN en SQL

Tipo	Descripción
INNER JOIN	Devuelve solo filas con coincidencia en ambas tablas
LEFT JOIN	Devuelve todas las filas de la izquierda + coincidencias de la derecha
RIGHT JOIN	Devuelve todas las filas de la derecha + coincidencias de la izquierda
FULL OUTER JOIN	Devuelve todas las filas de ambas tablas
CROSS JOIN	Producto cartesiano (cada fila con cada fila)
SELF JOIN	Una tabla se une consigo misma

5.2 Ejemplos Prácticos de JOIN

Ejemplo Práctico

INNER JOIN - Empleados con sus Departamentos:

```
1 SELECT e.nombre, e.salario, d.nombre as departamento
2 FROM empleados e
3 INNER JOIN departamentos d ON e.depto_id = d.id;
```

LEFT JOIN - Todos los Empleados (incluyendo sin departamento):

```
1 SELECT e.nombre, d.nombre as departamento
2 FROM empleados e
3 LEFT JOIN departamentos d ON e.depto_id = d.id;
```

5.3 Optimización de JOINS

Mejor Práctica

Mejores Prácticas para JOINS Eficientes:

- Indexar columnas usadas en condiciones JOIN
- Usar tipos de datos coincidentes en columnas de join
- Filtrar datos antes del JOIN cuando sea posible
- Preferir INNER JOIN sobre OUTER JOIN si no se necesitan NULLs
- Evitar JOINS en tablas muy grandes sin índices apropiados

6 Subconsultas

Definición

Una **subconsulta** es una consulta SQL anidada dentro de una consulta más grande. La subconsulta se ejecuta primero y su resultado se pasa a la consulta externa.

6.1 Tipos de Subconsultas

Table 8: Tipos de Subconsultas

Tipo	Retorna	Ejemplo
Escalar	Un valor	WHERE salario > (SELECT AVG(salario) ...)
Columna	Una columna	WHERE depto_id IN (SELECT id ...)
Fila Única	Una fila	WHERE (id, salario) = (SELECT ...)
Múltiples Filas	Varias filas	WHERE salario IN (SELECT ...)
Correlacionada	Ref. externa	WHERE ... (SELECT ... WHERE e2.depto = e1.depto)

Ejemplo Práctico

Subconsulta Correlacionada - Empleados con Salario Superior al Promedio de su Departamento:

```

1 SELECT e1.nombre, e1.salario, e1.depto_id
2 FROM empleados e1
3 WHERE e1.salario > (
4     SELECT AVG(e2.salario)
5     FROM empleados e2
6     WHERE e2.depto_id = e1.depto_id
7 );

```

6.2 Subconsultas vs JOINS

Table 9: Comparativa: Subconsultas vs JOINS

Subconsultas	JOINS
Más legibles para comparaciones simples	Generalmente más eficientes
Ideales para funciones agregadas	Preferibles para grandes volúmenes de datos
Pueden ser correlacionadas (más lentas)	Mejor optimización por el motor

7 Funciones SQL

7.1 Funciones de Cadena

Table 10: Funciones de Cadena Comunes

Función	Descripción	Ejemplo	Resultado
CONCAT()	Une cadenas	CONCAT('Hola', 'Mundo')	Hola Mundo
UPPER()	Mayúsculas	UPPER('hola')	HOLA
LOWER()	Minúsculas	LOWER('HOLA')	hola
LENGTH()	Longitud	LENGTH('hola')	4
SUBSTRING()	Extrae parte	SUBSTRING('hola', 1, 2)	ho
TRIM()	Sin espacios	TRIM(' hola ')	hola
REPLACE()	Reemplaza	REPLACE('hola', 'a', 'e')	hole

7.2 Funciones Numéricas

Table 11: Funciones Numéricas Comunes

Función	Descripción	Ejemplo
ROUND()	Redondea	ROUND(3.1416, 2) = 3.14
CEIL()	Redondea arriba	CEIL(3.2) = 4
FLOOR()	Redondea abajo	FLOOR(3.8) = 3
ABS()	Valor absoluto	ABS(-5) = 5
MOD()	Módulo (resto)	MOD(10, 3) = 1
POWER()	Potencia	POWER(2, 3) = 8

7.3 Funciones de Fecha y Hora

Table 12: Funciones de Fecha y Hora

Función	Descripción	Ejemplo
NOW()	Fecha y hora actual	NOW()
CURDATE()	Fecha actual	CURDATE()
DATE_ADD()	Suma intervalo	DATE_ADD(NOW(), INTERVAL 7 DAY)
DATEDIFF()	Diferencia en días	DATEDIFF(fecha1, fecha2)
YEAR()	Extrae año	YEAR(fecha)
MONTH()	Extrae mes	MONTH(fecha)
DAY()	Extrae día	DAY(fecha)

7.4 Funciones de Ventana (Window Functions)

Ejemplo Práctico

Funciones de Ventana para Análisis Avanzado:

```
1  -- ROW_NUMBER: Numerar filas
2  SELECT nombre, salario,
3         ROW_NUMBER() OVER (ORDER BY salario DESC) as ranking
4  FROM empleados;
5
6  -- LAG: Obtener valor anterior
7  SELECT nombre, salario,
8         LAG(salario) OVER (ORDER BY salario) as
9         salario_anterior
10 FROM empleados;
11
12 -- SUM acumulativo
13 SELECT nombre, salario,
14        SUM(salario) OVER (ORDER BY fecha_contratacion) as
15        acumulado
16 FROM empleados;
```

8 Vistas e Índices

8.1 Vistas (Views)

Definición

Una **vista** es una tabla virtual basada en una consulta almacenada. No almacena datos físicamente, sino que los recupera dinámicamente cuando se consulta.

Ejemplo Práctico

Creación y Uso de Vistas:

```
1  -- Crear vista
2  CREATE VIEW vista_empleados AS
3  SELECT e.id, e.nombre, e.salario, e.email, d.nombre as
4         departamento
5  FROM empleados e
6  JOIN departamentos d ON e.depto_id = d.id;
7
8  -- Usar vista
9  SELECT * FROM vista_empleados WHERE salario > 50000;
10
11 -- Actualizar vista
12 CREATE OR REPLACE VIEW vista_empleados AS
13 SELECT e.id, e.nombre, e.salario, e.email,
14        d.nombre as departamento, d.ubicacion
15 FROM empleados e
16 JOIN departamentos d ON e.depto_id = d.id;
17
18 -- Eliminar vista
19 DROP VIEW vista_empleados;
```

Beneficios de las Vistas:

- **Seguridad:** Oculta columnas sensibles
- **Simplificación:** Abstrae consultas complejas
- **Consistencia:** Centraliza lógica de consulta
- **Compatibilidad:** Mantiene interfaz ante cambios

8.2 Índices (Indexes)

Definición

Un **índice** es una estructura que mejora la velocidad de recuperación de datos, similar al índice de un libro.

Ejemplo Práctico

Gestión de Índices:

```
1  -- Crear ndice simple
2  CREATE INDEX idx_empleados_nombre ON empleados(nombre);
3
4  -- Crear ndice compuesto
5  CREATE INDEX idx_empleados_depto_salario
6  ON empleados(depto_id, salario);
7
8  -- Crear ndice nico
9  CREATE UNIQUE INDEX idx_empleados_email ON empleados(email);
10
11 -- Ver ndices (MySQL)
12 SHOW INDEX FROM empleados;
13
14 -- Eliminar ndice
15 DROP INDEX idx_empleados_nombre ON empleados;
```

Mejor Práctica

Cuándo Usar Índices:

Usar índices en:

- Columnas en cláusulas WHERE
- Columnas en condiciones JOIN
- Columnas en ORDER BY y GROUP BY
- Claves foráneas

Evitar índices en:

- Tablas pequeñas (< 1000 filas)
- Columnas con pocos valores únicos
- Columnas frecuentemente actualizadas

9 Stored Procedures y Triggers

9.1 Stored Procedures

Definición

Un **stored procedure** es un bloque de código SQL reutilizable almacenado en la base de datos que puede aceptar parámetros y ejecutar lógica compleja.

Ejemplo Práctico

Creación y Ejecución de Stored Procedures:

```

1 DELIMITER //
2 CREATE PROCEDURE obtener_empleados_por_depto(IN depto_id_param
3     INT)
4 BEGIN
5     SELECT e.nombre, e.salario, d.nombre as departamento
6     FROM empleados e
7     JOIN departamentos d ON e.depto_id = d.id
8     WHERE e.depto_id = depto_id_param;
9 END //
10 DELIMITER ;
11
12 -- Ejecutar
13 CALL obtener_empleados_por_depto(2);

```

Ventajas de Stored Procedures:

- **Reutilización:** Código escrito una vez, usado múltiples veces
- **Rendimiento:** Ejecución compilada y cacheada
- **Seguridad:** Control de acceso granular
- **Mantenibilidad:** Cambios centralizados

9.2 Triggers

Definición

Un **trigger** (disparador) es un procedimiento almacenado que se ejecuta automáticamente cuando ocurre un evento específico (INSERT, UPDATE, DELETE) en una tabla.

Table 13: Tipos de Triggers

Tipo	Evento	Uso Principal
BEFORE INSERT	Antes de insertar	Validación de datos
AFTER INSERT	Después de insertar	Logging, actualizaciones
BEFORE UPDATE	Antes de actualizar	Validación, transformación
AFTER UPDATE	Después de actualizar	Auditoría, sincronización
BEFORE DELETE	Antes de eliminar	Prevención, backup
AFTER DELETE	Después de eliminar	Logging, limpieza

Ejemplo Práctico

Trigger de Auditoría - Registrar Cambios de Salario:

```
1  -- Tabla de auditoría
2  CREATE TABLE empleados_audit (
3      id INT AUTO_INCREMENT PRIMARY KEY,
4      empleado_id INT,
5      accion VARCHAR(20),
6      salario_anterior DECIMAL(10,2),
7      salario_nuevo DECIMAL(10,2),
8      fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP
9  );
10
11  -- Trigger
12  DELIMITER //
13  CREATE TRIGGER audit_salario_cambio
14  AFTER UPDATE ON empleados
15  FOR EACH ROW
16  BEGIN
17      IF OLD.salario != NEW.salario THEN
18          INSERT INTO empleados_audit (empleado_id, accion,
19              salario_anterior, salario_nuevo)
20              VALUES (OLD.id, 'UPDATE', OLD.salario, NEW.salario);
21      END IF;
22  END //
23  DELIMITER ;
```

10 Optimización de Consultas

10.1 Principios Fundamentales

Información Clave

Estadísticas de Rendimiento (2025):

- **63%** de problemas de rendimiento se deben a SQL ineficiente
- Solo **7%** de consultas consumen más del **70%** de recursos
- Índices apropiados pueden mejorar rendimiento **20x** o más

Table 14: Principios de Optimización SQL

Principio	Descripción
Indexación estratégica	Indexar columnas en WHERE, JOIN, ORDER BY
SELECT específico	Evitar SELECT *, especificar columnas necesarias
Filtrar temprano	Aplicar WHERE lo antes posible
Evitar funciones en índices	No usar funciones en columnas indexadas
JOINS eficientes	Unir en columnas indexadas con tipos coincidentes
EXISTS sobre IN	EXISTS es más eficiente para verificaciones
UNION ALL sobre UNION	UNION ALL omite verificación de duplicados

10.2 Técnicas de Optimización Prácticas

Ejemplo Práctico

Antes y Después de Optimización:

Consulta Ineficiente:

```
1 SELECT * FROM empleados e, departamentos d
2 WHERE e.depto_id = d.id
3     AND YEAR(e.fecha_contratacion) = 2023;
```

Consulta Optimizada:

```
1 -- índices creados
2 CREATE INDEX idx_empleados_depto ON empleados(depto_id);
3 CREATE INDEX idx_empleados_fecha ON empleados(
4     fecha_contratacion);
5
6 -- Consulta reescrita
7 SELECT e.id, e.nombre, e.salario, d.nombre as departamento
8 FROM empleados e
9 INNER JOIN departamentos d ON e.depto_id = d.id
10 WHERE e.fecha_contratacion >= '2023-01-01'
    AND e.fecha_contratacion < '2024-01-01';
```

Mejora obtenida: 70% menos datos transferidos, 20x más rápido

10.3 EXPLAIN y Planes de Ejecución

```
1 -- Analizar plan de ejecución
2 EXPLAIN SELECT e.nombre, d.departamento
3 FROM empleados e
4 JOIN departamentos d ON e.depto_id = d.id
5 WHERE e.salario > 50000;
```

Table 15: Columnas Clave de EXPLAIN

Columna	Descripción
type	Tipo de join (ALL = escaneo completo, ref = con índice)
key	Índice utilizado
rows	Filas estimadas a examinar
Extra	Información adicional (Using index, Using where, etc.)

11 Transacciones y Propiedades ACID

Definición

Una **transacción** es una secuencia de operaciones que se ejecutan como una unidad atómica, garantizando integridad incluso en caso de fallos.

11.1 Propiedades ACID

Table 16: Propiedades ACID de Transacciones

Propiedad	Descripción
Atomicidad	Todo o nada: o se ejecutan todas las operaciones o ninguna
Consistencia	La transacción mantiene la integridad de la base de datos
Aislamiento	Transacciones concurrentes no interfieren entre sí
Durabilidad	Una vez confirmada, la transacción persiste aunque haya fallo

11.2 Ejemplo de Transacción Bancaria

Ejemplo Práctico

Transferencia con Control de Transacción:

```

1 START TRANSACTION;
2
3 -- Debitar de cuenta A
4 UPDATE cuentas SET saldo = saldo - 100 WHERE id = 1;
5
6 -- Acreditar a cuenta B
7 UPDATE cuentas SET saldo = saldo + 100 WHERE id = 2;
8
9 -- Verificar saldo (debe ser igual)
10 -- Si hay error:
11 -- ROLLBACK;
12
13 -- Si todo est correcto:
14 COMMIT;
```

11.3 Niveles de Aislamiento

Table 17: Niveles de Aislamiento de Transacciones

Nivel	Dirty Read	Non-Repeatable	Phantom
READ UNCOMMITTED	Posible	Posible	Posible
READ COMMITTED	No	Posible	Posible
REPEATABLE READ	No	No	Posible
SERIALIZABLE	No	No	No

12 SQL Avanzado

12.1 Common Table Expressions (CTE)

Ejemplo Práctico

CTE Simple y Recursivo:

```

1  -- CTE simple
2  WITH empleados_senior AS (
3      SELECT * FROM empleados WHERE salario > 60000
4  )
5  SELECT nombre, salario FROM empleados_senior;
6
7  -- CTE recursivo (jerarquía organizacional)
8  WITH RECURSIVE jerarquia AS (
9      SELECT id, nombre, jefe_id, 0 as nivel
10     FROM empleados WHERE jefe_id IS NULL
11     UNION ALL
12     SELECT e.id, e.nombre, e.jefe_id, j.nivel + 1
13     FROM empleados e
14     JOIN jerarquia j ON e.jefe_id = j.id
15 )
16 SELECT * FROM jerarquia ORDER BY nivel;

```

12.2 PIVOT - Transformar Filas a Columnas

```

1  -- PIVOT manual
2  SELECT departamento,
3      MAX(CASE WHEN ano = 2023 THEN ventas END) as ventas_2023,
4      MAX(CASE WHEN ano = 2024 THEN ventas END) as ventas_2024
5  FROM ventas
6  GROUP BY departamento;

```

13 Mejores Prácticas y Seguridad

13.1 Diseño de Base de Datos

Mejor Práctica

Principios de Diseño:

- Normalizar hasta 3NF (Tercera Forma Normal)
- Usar nombres descriptivos y consistentes
- Documentar esquemas y relaciones
- Definir constraints apropiados (PK, FK, UNIQUE, CHECK)
- Planear índices desde el diseño inicial

13.2 Seguridad SQL

Table 18: Prácticas de Seguridad SQL

Práctica	Descripción
Principio de menor privilegio	Otorgar solo permisos estrictamente necesarios
Prepared statements	Prevenir SQL injection usando parámetros
Enmascaramiento de datos	Ocultar datos sensibles en consultas
Auditoría regular	Revisar logs y accesos periódicamente
Cifrado	Cifrar datos sensibles en reposo y tránsito

Ejemplo Práctico

Prevenir SQL Injection:

Vulnerable:

```

1  -- NUNCA hacer esto!
2  SET @query = CONCAT('SELECT * FROM empleados WHERE id = ',
3  user_input);
4  PREPARE stmt FROM @query;

```

Seguro con Prepared Statement:

```

1  PREPARE stmt FROM 'SELECT * FROM empleados WHERE id = ?';
2  SET @id = 1;
3  EXECUTE stmt USING @id;
4  DEALLOCATE PREPARE stmt;

```

14 Conclusiones

SQL continúa siendo la piedra angular del manejo de datos en 2026. Su evolución constante, compatibilidad universal y poder expresivo lo mantienen como habilidad esencial para profesionales de tecnología.

Información Clave

Puntos Clave para Recordar:

- SQL es **declarativo**: especificas QUÉ quieres, no CÓMO obtenerlo
- La **optimización** requiere comprensión del motor de base de datos
- **Índices** son cruciales para rendimiento pero afectan escrituras
- **Transacciones ACID** garantizan integridad en operaciones críticas
- La **seguridad** debe considerarse desde el diseño inicial

Recomendación Final: La maestría en SQL se logra mediante práctica constante. Comience con consultas básicas, progrese a JOINS y subconsultas, y eventualmente domine optimización y características avanzadas.

A Recursos de Aprendizaje

A.1 Plataformas Recomendadas

Table 19: Plataformas para Aprender SQL

Plataforma	Tipo	Costo
SQLZoo	Interactivo	Gratis
Mode Analytics SQL Tutorial	Tutorial	Gratis
LeetCode SQL	Práctica	Gratis/Premium
DataCamp	Cursos	Subscription
Coursera	Certificaciones	Variable

A.2 Ruta de Aprendizaje Sugerida

- **Semanas 1-2:** Sintaxis básica (SELECT, WHERE, ORDER BY)
- **Semanas 3-4:** JOINS y funciones de agregación
- **Semanas 5-6:** Subconsultas y funciones avanzadas
- **Semanas 7-8:** Vistas, índices, optimización
- **Semanas 9-10:** Stored procedures, triggers, transacciones
- **Semanas 11-12:** Proyectos prácticos completos

B Referencias Bibliográficas

- GeeksforGeeks. (2025). *SQL Roadmap: A Complete Guide*
- TxMinds. (2026). *High-Performance SQL: 12 Proven Query Optimization Techniques*
- Oboe. (2026). *SQL Joins Explained*
- w3resource. (2026). *SQL Subqueries*
- Microsoft. (2025). *SQL Server Documentation*
- PostgreSQL Global Development Group. (2025). *PostgreSQL Documentation*